Eval Engineering

Measuring a non-deterministic system

Criteria · Datasets · Graders · Judges
Agents · RAG · Traces · Error bars · Gates · Production

Part 1: Foundations

What an eval is · Defining correct · Datasets

Part 2: Grading

Graders · LLM-as-judge and calibration

Part 3: Applying it

Agents · RAG and grounding · Tracing

Part 4: Deciding

Reading results · CI gates · Production · Failure modes

Notes: ai-engineering/06-eval

PART 1

Foundations

What an eval is · Defining correct · Datasets

The thesis

Six parts, many names

The core flow

Tests gate merges, evals measure rates

Capability vs. regression suites

Write success down before any code

Find failure modes by reading

The failure taxonomy, ordered by stage

Start small and real, grow by stage

Is this row fair for this gate?

What public benchmarks teach private suites

The thesis

> **An eval is the definition of "better" for your system, written down so a machine can check it.**
>
> *Without it, every change to a prompt, model, tool or retrieval config is a guess.*

Source: notes/01-what-an-eval-is.md

Six parts, many names

Anthropic, LangSmith, OpenAI and Inspect use different words for the same parts.

Term	What it is	Also called
Task	One test: an input plus success criteria	case, example, row
Trial	One attempt at a task. Outputs vary, so several per task	run, epoch
Trajectory	The full record: messages, tool calls, intermediate results	trace, transcript
Outcome	The end state: the answer, the file system, the DB row	final response
Grader	Code, a model or a person scoring one aspect of a trial	evaluator, judge, metric
Suite	Tasks run together for a system-level picture	eval set, benchmark

> **Inspect's split: a Task = dataset + solver (the system) + scorer. Keep the three swappable.**

Source: notes/01-what-an-eval-is.md

The core flow

Every chapter is one arrow



Chapters 02–12 are each about doing one of these arrows well.

Source: notes/01-what-an-eval-is.md

Tests gate merges, evals measure rates

Draw the line by determinism.

Unit test

- Deterministic behavior: rendering, schemas, retries, routing tables
- Asserts exact values. Pass or fail
- Mock the model here

"retry fires 3 times on 429"

Eval

- Non-deterministic behavior: quality, grounding, task success
- Many tasks, many trials. A rate with an error bar
- **Never** mock the model: you'd measure the mock

"grounded answer rate ≥ 0.90 on the golden set"

Never assert an exact LLM string in a unit test. It flakes, then gets loosened until it checks nothing.

Source: notes/01-what-an-eval-is.md

Capability vs. regression suites

Two questions, two pass rates

	Capability eval	Regression eval
Question	What can it do?	Does it still do what it used to?
Expected pass rate	Low to medium. Room to climb	Near **100%**
Tasks come from	The frontier: hard and new cases	Mastered tasks + every fixed bug
A drop means	Little on its own	Something broke. Block the release
Runs	On demand	Every change, in CI

> **Tasks graduate. Once a capability task passes reliably, it moves to the regression suite.**

Source: notes/01-what-an-eval-is.md

Write success down before any code

Question	Bad answer	Good answer
What does the user need?	A helpful answer	The steps to change their payout account, in their language, citing the article
What must never happen?	Nothing bad	Inventing a menu path. Leaking another customer's data
Unit of success?	It went well	Per turn: grounded + complete. Per conversation: resolved in ≤ 6 turns
Cost of being wrong?	—	Wrong menu path = a ticket. Wrong tax answer = a fine

> **The two-expert test: if two experts wouldn't reach the same verdict, fix the criteria, not the grader.**

Source: notes/02-defining-correct.md

Find failure modes by reading

Error analysis (Husain & Shankar)

1. **Collect ~100 traces** — Real if you have them, synthetic if you don't yet.
2. **Open coding** — A free-text note on the **first** thing that went wrong. No categories yet.
3. **Axial coding** — Group the notes into categories. Expect **5–15**.
4. **Count** — Rank by frequency × cost. That ranking is your eval roadmap.
5. **Repeat to saturation** — Until new traces stop producing new categories, usually 50–100.

> **Don't build a grader for a failure mode you haven't seen.**

Source: notes/02-defining-correct.md

The failure taxonomy, ordered by stage

Stable names, one label per fix, earliest failure wins.

Stage	Labels
Corpus	`source_missing` · `url_unmapped` · `topic_proxy`
Index	`index_lag` · `chunk_fragmented`
Retrieval	`rank_miss` · `query_rewrite_miss` · `reranker_regression`
Eval infrastructure	`context_missing`
Generation	`citation_hallucination` · `unsupported_claim` · `incomplete_answer`
Behavior	`wrong_escalation` · `refusal` · `schema_violation`

> **Give the eval's own failures a label, so eval bugs aren't counted as system bugs.**

Source: notes/02-defining-correct.md

Start small and real, grow by stage

Stage	Size	Purpose
Example suite	5–20	Eyeball diffs while iterating on a prompt
First golden set	**20–50**	Real failures, hand-reviewed. Catches big regressions
Working set	100–300	±5–8 point error bars on a pass rate
LLM-graded sample	50–200	Cost-capped subset for expensive judges
Large harness	1,000+	Small effects. Needs synthetic or logged data

> **Grow to the next stage when the current one can't separate two versions you need to tell apart.**

Source: notes/03-eval-datasets.md

Is this row fair for this gate?

Each pool is a subset of the one above it.

narrower, fairer for later gates

1. **all_gt** — Every golden row. Fair for demand analysis.
2. **source_mapped_gt** — Expected source maps to a known doc. Fair for corpus coverage.
3. **retrieval_eligible_gt** — That source is actually indexed. Fair for retrieval metrics.
4. **generation_eligible_gt** — Retrieval found it. Fair for answer quality.
5. **quality_sample** — Deterministic, cost-capped sample. Fair for LLM judges.

> **A downstream gate never silently scores rows an upstream gate marked ineligible.**

Source: notes/03-eval-datasets.md

What public benchmarks teach private suites

Budget for rejection

SWE-bench Verified: 93 developers reviewed tasks. Only **500 of 2,294** survived.

Fixed it, broke nothing

FAIL_TO_PASS tests prove the fix. PASS_TO_PASS tests prove no regression.

Difficulty as diagnosis

GAIA L1: humans 93.9%, GPT-4 30.3%. L3: 87.3% vs **0%**.

Fresh parameters

τ²-bench and AndroidWorld fill templates with new values every run. Memorized paths won't replay.

Canaries

Terminal-Bench embeds a unique GUID. If a model can produce it, the set leaked.

So…

Split dev and held-out, version the dataset, record the version with every result.

Source: notes/03-eval-datasets.md

PART 2

Grading

Graders · LLM-as-judge and calibration

Code first, judge above it, humans calibrate

One contract for every grader

Which graders may block a release

Don't let a composite hide a regression

One judge, one criterion, reason before verdict

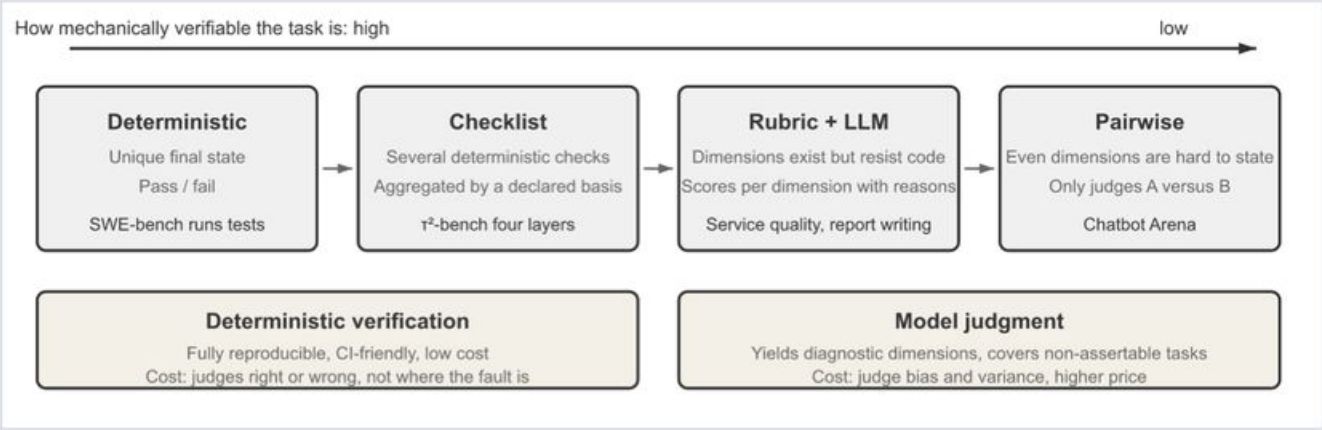
Known judge biases and their fixes

Calibrate before you gate

Report the two error rates separately

Pairwise votes become a ranking

Code first, judge above it, humans calibrate



Verification modes, deterministic to model judgment (Li, AI Agents in Depth, Fig. 7-4)

- **Code:** deterministic, free, hard to argue with
- **Separated evaluator:** runs the output, doesn't just read it
- **LLM judge:** semantic quality at volume. Biased, needs calibration
- **Humans:** 5–10% of graded rows, to **calibrate** the judge

Layers, not alternatives. Run code first and stop early, so judge budget goes where judgment is needed.

Source: notes/04-graders.md

One contract for every grader

PYTHON

```python
class Grader(Protocol):
    name: str
    version: str
    # e.g. {"retrieved_context"}: skip if missing
    requires: set[str]

    def grade(self, task, output,
              trajectory=None,
              context=None) -> GraderResult: ...

# GraderResult: score, passed, reasoning,
#   dimensions, labels, grader_version
```

- **`requires`:** skip with `context_missing`, don't guess
- **`reasoning`:** the judge's explanation, or the failing check
- **`labels`:** taxonomy labels this trial earned
- **`grader_version`:** every score traceable

> **Same input, same output shape. Graders become swappable, composable and reportable the same way.**

Source: notes/04-graders.md

Which graders may block a release

State	Meaning	Can gate?
Heuristic	Deterministic code check	**Yes**
Calibrated	LLM judge with evidence it agrees with humans	Yes, at the calibrated threshold
Experimental	New or uncalibrated judge	**No.** Tracking only

> **A threshold change without fresh calibration evidence moves a grader back to experimental.**

Source: notes/04-graders.md

Don't let a composite hide a regression

Weighted composite alone

- +5 relevance, –5 grounding = flat score
- Worse for users who need correct answers
- A fluent answer with a made-up fact still scores well

Hybrid with a veto

- Must-pass gates: schema valid, **no** citation hallucination
- Then 0.5 grounding + 0.3 completeness + 0.2 relevance ≥ 0.75
- Hallucination is a **veto**: fires → total is 0

Prefer binary pass/fail per dimension over 1–5 scales. "Is this a 3 or a 4?" has no stable answer.

Source: notes/04-graders.md

One judge, one criterion, reason before verdict

TEXT

```
You are evaluating whether an answer to a query
is clear.
[BEGIN DATA]
Query: {query}
Answer: {response}
[END DATA]
"clear": easy to follow, answers the query.
"unclear": confusing, off-topic, hard to act on.

EXPLANATION: <step-by-step reasoning>
LABEL: "clear" or "unclear"
```

- Grade a third party's work, never "your own"
- Precise PASS and FAIL definitions
- 2–4 few-shot cases near the boundary
- Delimited inputs; critique, then a fixed-format label
- A `CANNOT_GRADE` escape, or the judge guesses

Parse strictly. An unparseable verdict is a grader error, not a FAIL.

Source: notes/05-llm-as-judge.md

Known judge biases and their fixes

Zheng et al. 2023 (MT-Bench)

Bias	What happens	Mitigation
Position	Pairwise judges favor one slot	Run both orders. A win must hold in both
Verbosity	Longer answers score higher	Rubric says length isn't quality. Track score vs. length
Self-preference	Favors its own model family	Judge with a different family
Leniency	Borderline cases pass	Explicit FAIL conditions. Tricky-FAIL few-shots
Format / authority	Confident tone, markdown, citations look right	Check citations in code
Limited capability	Can't verify what it can't do	A reference, a code check, or a stronger judge

Once these are controlled, strong judges agree with humans about as often as humans agree with each other (over 80%).

Source: notes/05-llm-as-judge.md

Calibrate before you gate

1. **Get human labels** — **100–200** outputs labeled PASS/FAIL by your domain expert.
2. **Split** — Train (few-shots), dev (iterate), test (measure once).
3. **Run on dev** — Build the confusion matrix against the human labels.
4. **Read disagreements** — Each is a prompt problem, a criteria problem, or a hard case.
5. **Measure on test once** — That's the reported number. Record judge model and prompt version.

> **An uncalibrated judge never gates a release.**

Source: notes/05-llm-as-judge.md

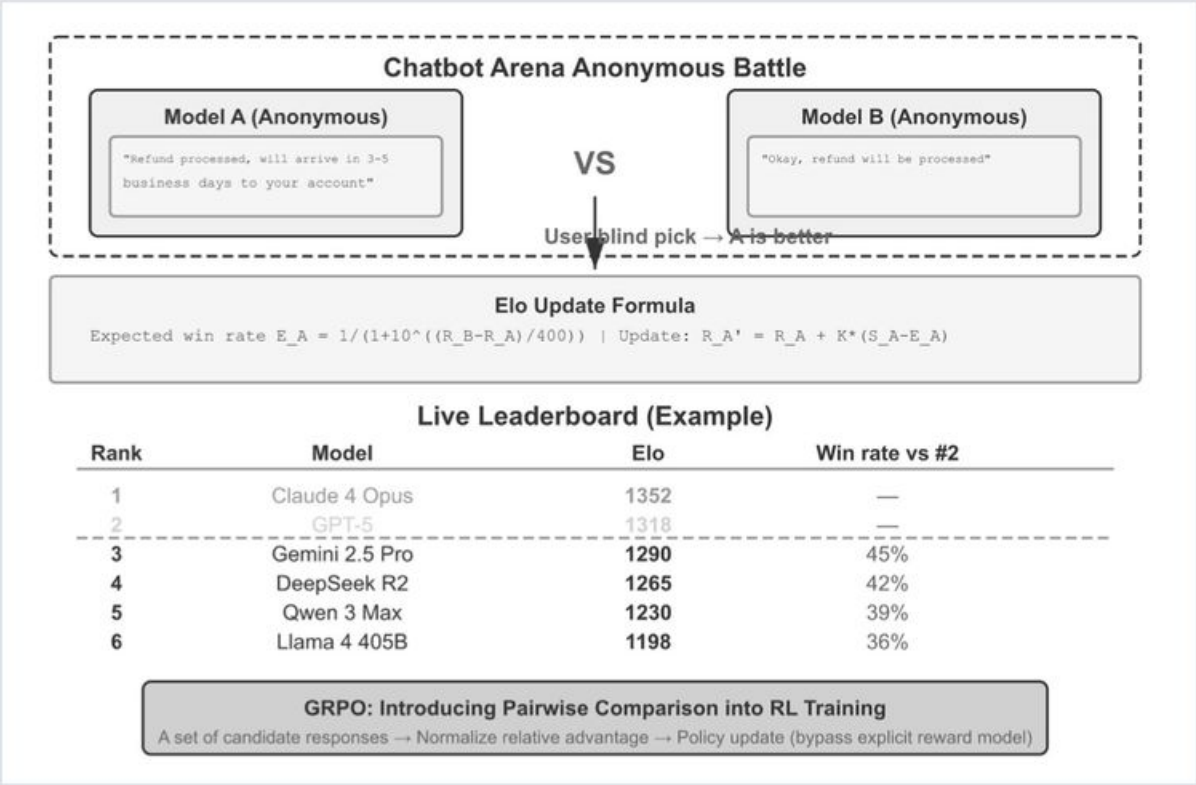
Report the two error rates separately

Metric	Meaning	Matters for
TPR	Of human PASSes, how many the judge passed	Not blocking good releases
TNR	Of human FAILs, how many the judge failed	**Catching bad outputs**
Precision on PASS	Of judge PASSes, how many humans passed	Building golden labels
Cohen's κ	Agreement corrected for chance	Comparing to human–human
Score separation	Mean on human-PASS minus mean on human-FAIL	Does it discriminate at all?

> **Correct the pass rate: true ≈ (p̂ + TNR – 1) / (TPR + TNR – 1).**

Source: notes/05-llm-as-judge.md

Pairwise votes become a ranking



Elo rating from pairwise comparisons (Li, AI Agents in Depth, Fig. 7-6)

- Expected: $E = 1 / (1 + 10^{((R_B - R_A)/400)})$
- Update: $R \leftarrow R + K \cdot (S - E)$
- 1200 vs 1000 → expected win **76%**
- LLM voter: **swap the order**, count a win only if it holds both ways
- Check for cycles; Bradley–Terry assumes transitivity

Pairwise is easier for the judge than absolute scoring, but you can't monitor production with it.

Source: notes/05-llm-as-judge.md

PART 3

Applying it

Agents · RAG and grounding · Tracing

Outcome and trajectory fail independently

Three strictness levels for tool calls

One grader per component

The router judge, in code

pass@k rises, pass^k falls

Convergence is relative to what you observed

Why agent cost grows faster than step count

Score each RAG stage separately

Check grounding claim by claim

A trace is a tree of spans

Traces feed evals, evals feed datasets

Outcome and trajectory fail independently

Outcome	Trajectory	What happened	Outcome-only eval says
Pass	Pass	Worked	Pass
Pass	Fail	Right answer by an unsafe or wasteful path: 40 paid API calls	**Pass (wrong)**
Fail	Pass	Reasonable process, external failure: tool timeout	Fail (not the agent)
Fail	Fail	Broken	Fail

> **Grade the end state, not what the agent says. "I've issued your refund" with no refund row is the failure to catch.**

Source: notes/06-evaluating-agents.md

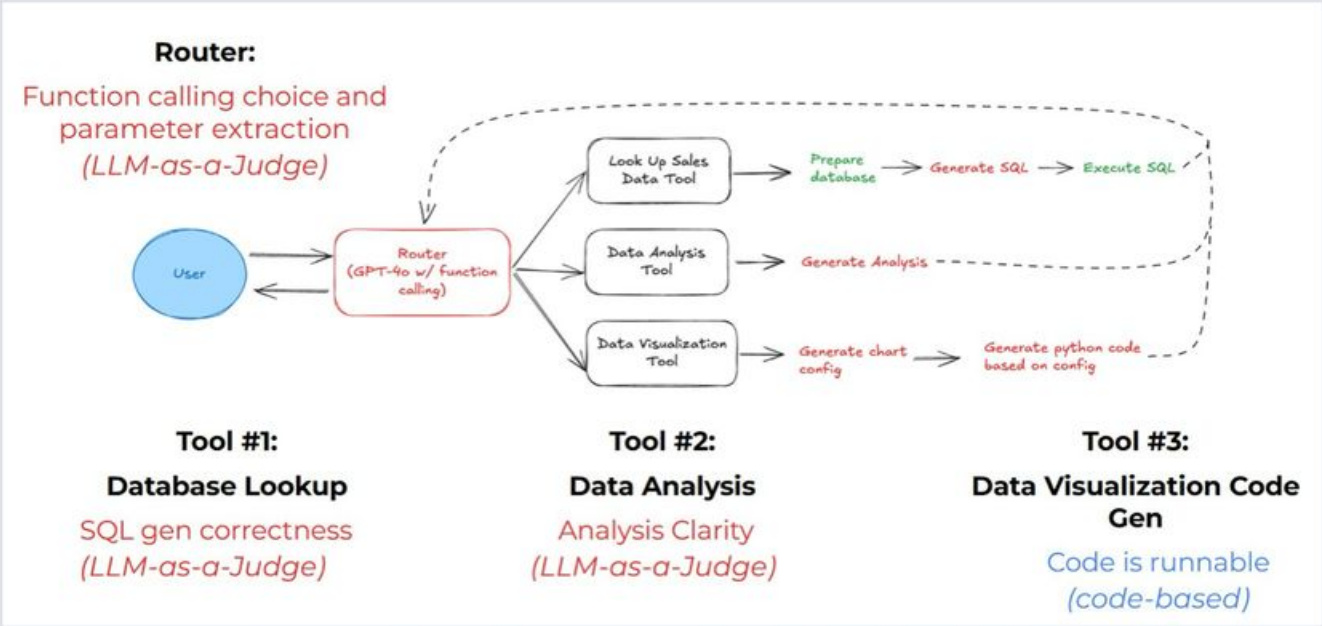
Three strictness levels for tool calls

Match mode	Passes when	Use for
Exact	Same tools, order and arguments	Strict protocols (auth before action)
In order	Expected calls in order, extras allowed	Required sequence with some freedom
Any order	All expected calls appear	Independent lookups
Subset / superset	Calls $\subseteq$ or $\supseteq$ an allowed set	"Never call outside this set"

> **No single correct path? Use invariants ("never issue_refund before lookup_order") or a trajectory judge.**

Source: notes/06-evaluating-agents.md

One grader per component



Router and skill evaluators (DeepLearning.AI & Arize, Lab 3)

- **Router:** right tool, right parameters (LLM judge)
- **SQL skill:** does the query answer the question (judge)
- **Analysis skill:** clear? entities correct? (two judges)
- **Visualization:** does the code run (code)
- **Path:** converged in few steps (code)

> **An end-to-end failure doesn't say which part broke. Component evals do.**

Source: notes/06-evaluating-agents.md

The router judge, in code

PYTHON

```python
q = SpanQuery().where("span_kind == 'LLM'").select(
    question="input.value", tool_call="llm.tools")
df = px.Client().query_spans(q, project_name=P)
df = df.dropna(subset=["tool_call"])

with suppress_tracing():
    ev = llm_classify(
        dataframe=df,
        template=TOOL_CALLING_PROMPT_TEMPLATE,
        rails=["correct", "incorrect"],
        model=OpenAIModel(model="gpt-4o"),
        provide_explanation=True)

px.Client().log_evaluations(SpanEvaluations(
    eval_name="Tool Calling Eval", dataframe=ev))
```

- Spans in, labels back onto the same spans
- `rails` snap the output to allowed labels
- **Moved since:** `phoenix.client.Client`, span annotations
- phoenix.evals 2.x: `create_classifier`, `LLM(provider=…)`

> **Grade the trace you already recorded. The score lives one click from the context.**

Source: notes/06-evaluating-agents.md

pass@k rises, pass^k falls

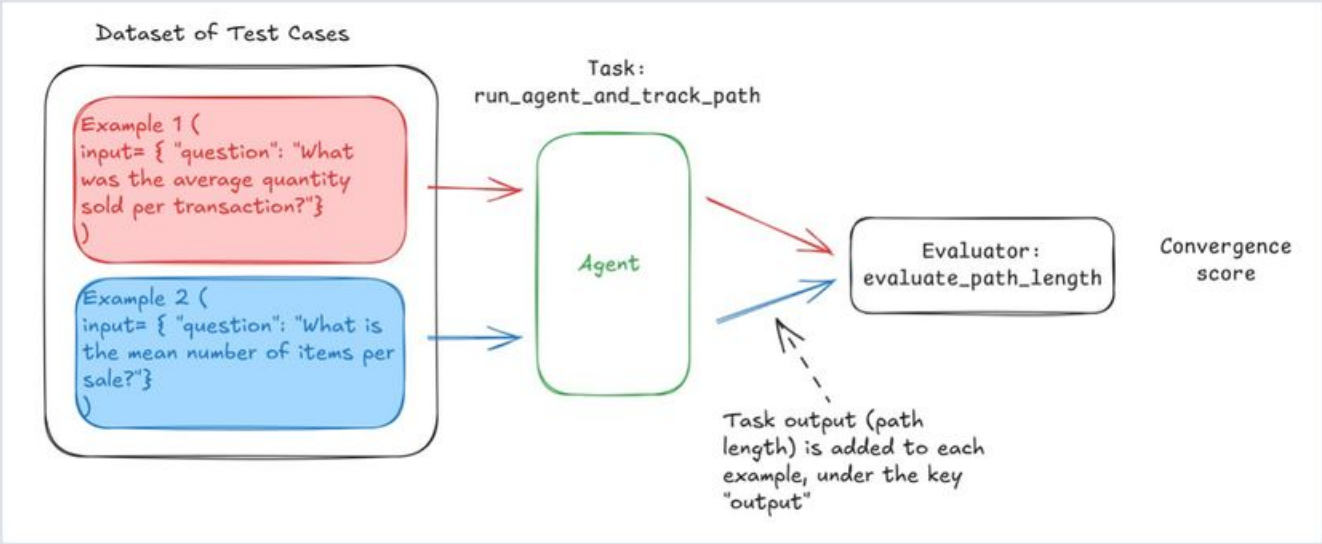
Run each task k times (3–10 is typical) and report rates.

Metric	Definition	Question	p = 0.75, k = 3
pass@k	At least one of k succeeds	Can it do this at all, with retries?	**0.98**
pass^k	All k succeed	Does it do this every time?	**0.42**
Best@k	Best score of k	What's the quality ceiling?	—

> **Customer-facing agents get one try per user. Report pass^k for them.**

Source: notes/06-evaluating-agents.md

Convergence is relative to what you observed



Convergence experiment (DeepLearning.AI & Arize, Lab 4)

- **17** paraphrases of one question
- Score = shortest run seen ÷ this run's length
- "Optimal" is the shortest **observed**, so a thrashing batch still scores near 1.0
- Counts messages, not tool calls
- Fix the reference length before gating on it

> **Log steps, tokens, latency and cost per success on every trial. Pass rates miss efficiency regressions.**

Source: notes/06-evaluating-agents.md

Why agent cost grows faster than step count

Li, AI Agents in Depth, Table 6-4

A three-turn refund agent: 9,500 input tokens, 470 output, **$0.022**. Every turn re-sends the context.

Thinking tokens

500–2,000 extra per turn can multiply cost **3–5×**. 2,000 at 50 tok/s is 40 s.

Big tool outputs

A 5,000-token page from turn 2 is billed again on every later turn.

Detours

Ten turns instead of three, and the context passes **20,000** tokens.

So…

Cap cost per task in the harness. Log thinking tokens separately. Report p95 latency, not the mean.

Source: notes/06-evaluating-agents.md

Score each RAG stage separately



- Pick **k** to match what enters the prompt. Recall@50 is irrelevant if 5 are passed
- Score generation only on **generation-eligible** rows
- Faithful is not correct: faithful-but-wrong points at the corpus

Source: notes/07-evaluating-rag-and-grounding.md

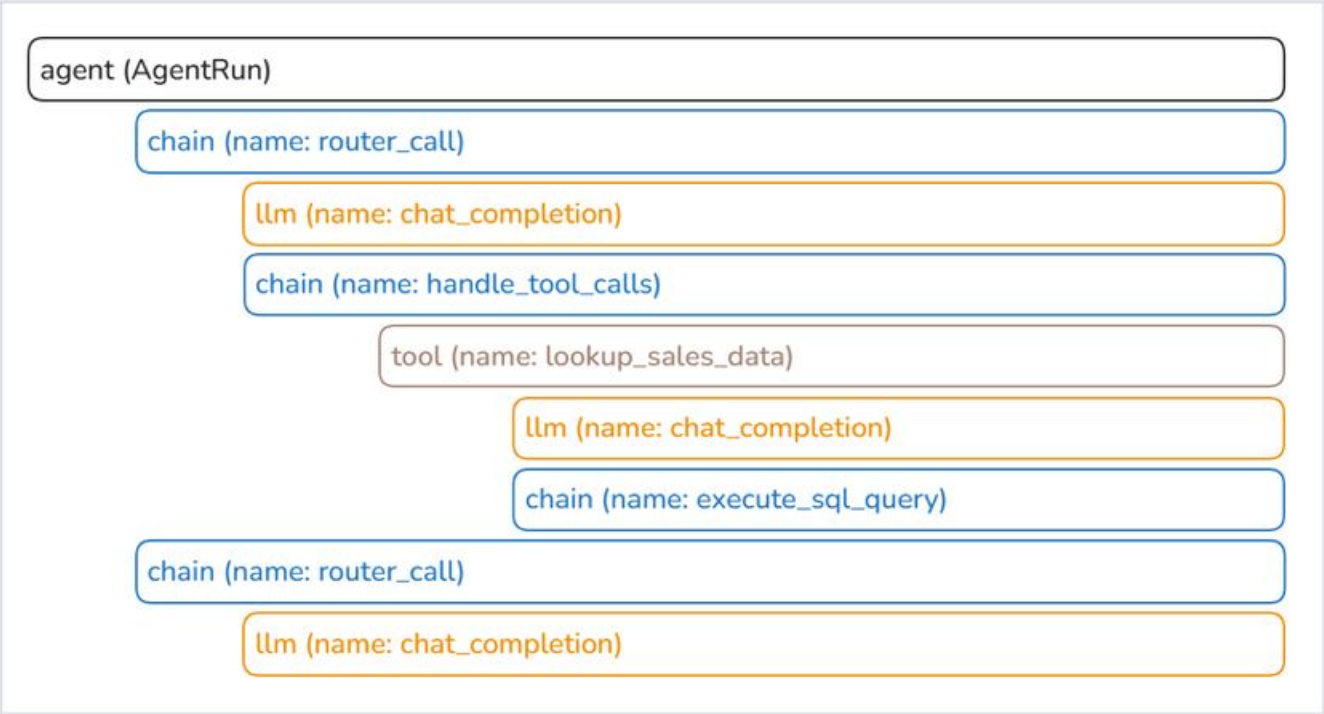
Check grounding claim by claim

1. **Extract claims** — Split the answer into atomic statements. "Settings → Payouts, click Edit, 2 days" is three.
2. **Verify each** — Against passage **text**: supported, contradicted or not found.
3. **Score** — Supported ÷ total. Report contradictions separately.
4. **Tier the claims** — Strict facts need near-verbatim support; paraphrase needs entailment; filler is exempt.

> **Pass the text, not the pointer. A grader handed URLs scores near zero, or from its own knowledge.**

Source: notes/07-evaluating-rag-and-grounding.md

A trace is a tree of spans



One agent run in Phoenix (DeepLearning.AI & Arize, Lab 2)

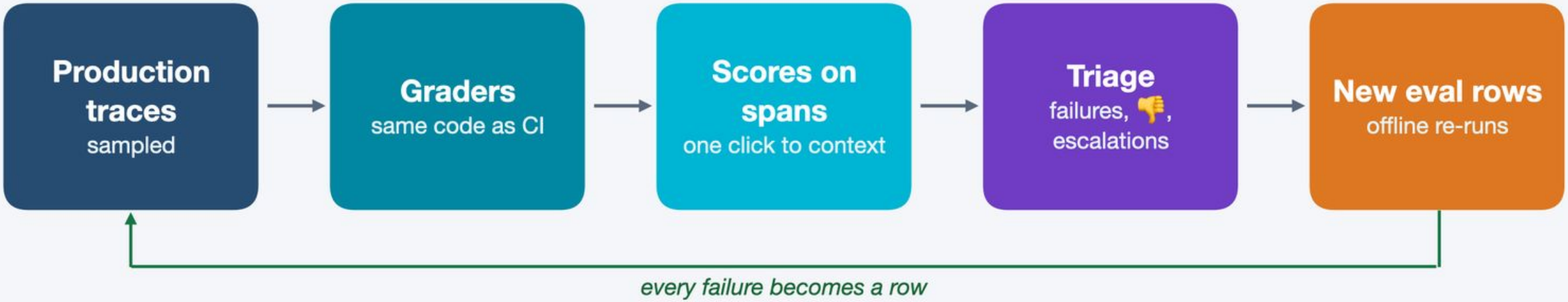
- **Model call:** model ID, full input, tokens, latency
- **Tool call:** name, arguments, result or error
- **Retrieval:** query, IDs **and text**, scores
- **Root:** prompt, model, config, index **versions**
- Auto-instrument LLM calls; add agent, chain, tool spans yourself

> **Final-output evals say that something failed. Traces say where.**

Source: notes/08-tracing-and-observability.md

Traces feed evals, evals feed datasets

The loop that makes traces pay off



Pick a tool (Phoenix, Langfuse, LangSmith, Braintrust) on four things: evals over stored traces, scores on spans, datasets from traces, side-by-side experiments.

Source: notes/08-tracing-and-observability.md

PART 4

Deciding

Reading results · CI gates · Production · Failure modes

Every number gets an error bar

Size the suite for the smallest change that matters

Compare versions on the same tasks

A worked example: 7 questions, one run

Slice before you conclude

Slice by difficulty, not just by topic

A report that supports a decision

Cheap suites on every change, expensive ones on a schedule

Run, then assert

Gate against a baseline, on the interval

Gates ordered by pipeline stage

The maturity ladder

Humans calibrate, judges cover

When a score drops, suspect the eval first

Optimize hard and you'll find the eval's gaps

Every number gets an error bar

SE = √(p(1 − p) / n). The 95% CI is p ± 1.96·SE.

Tasks	SE at p = 0.80	95% CI
20	0.089	62% – 98%
50	0.057	**69% – 91%**
100	0.040	72% – 88%
300	0.023	75% – 85%
1,000	0.013	77% – 83%

> **On 50 tasks, 80% and 72% are not distinguishable.**

Source: notes/09-reading-results.md

Size the suite for the smallest change that matters

Paired, 80% power, α = 0.05, pass rates around 70–85%.

Smallest difference that matters	Tasks needed
15 points	50–100
10 points	100–200
5 points	400–800
2 points	**2,500+**

> **A 50-task golden set is a smoke detector for big regressions. It can't tell you a tweak gained 3 points.**

Source: notes/09-reading-results.md

Compare versions on the same tasks

Paired differences (Miller, Anthropic 2024)

Two independent rates

- A at 84%, B at 85%, each with its own wide interval
- Task-to-task variation swamps the difference

The error bars overlap and you learn nothing

Per-task differences

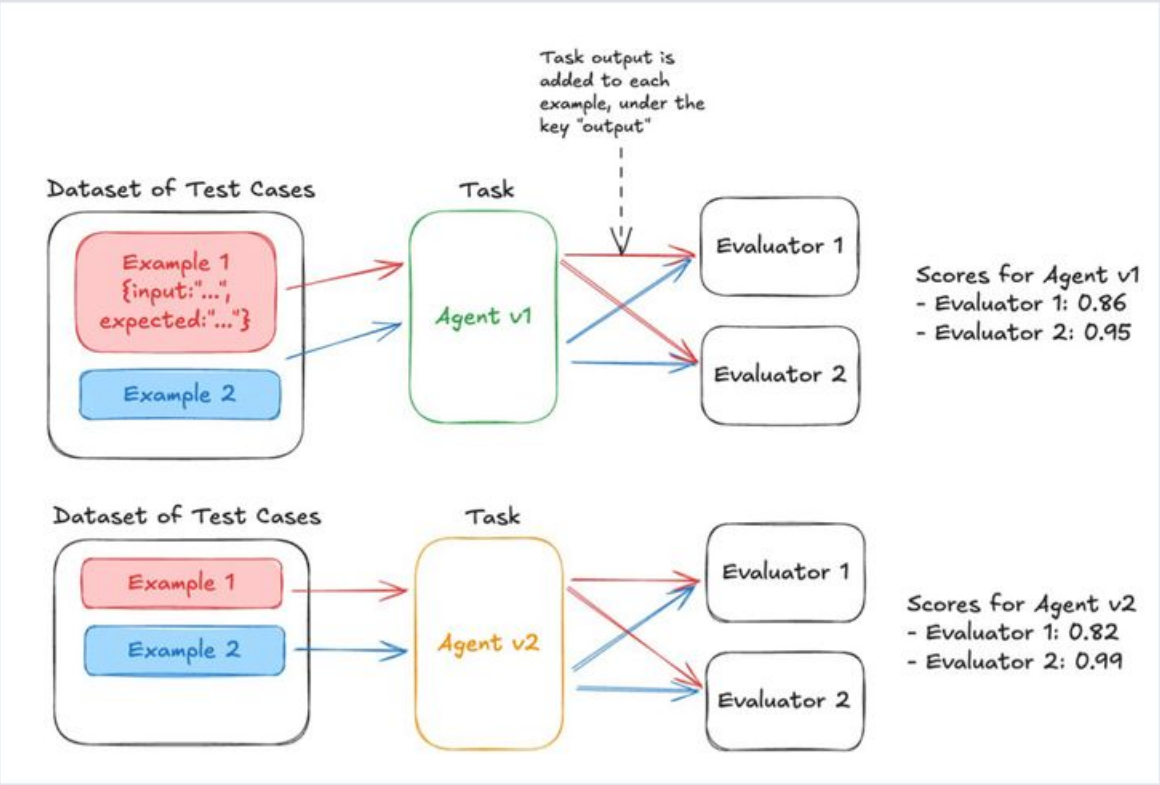
- Most tasks pass or fail on both, so differences are mostly 0
- SE(Δ) = sd(differences) / √n, much smaller
- McNemar: only the **discordant** tasks count
- B wins **14–6** is evidence. **11–9** is not

Pairing cuts variance for free

Run both versions on identical tasks and analyze per-task differences.

Source: notes/09-reading-results.md

A worked example: 7 questions, one run



v1 vs v2 experiment (DeepLearning.AI & Arize, Lab 5)

- v2 adds "Think before you respond." to the SQL prompt
- **n = 7**: one flipped task moves a rate **14 points**
- 0.82 vs 0.86 is noise. Pair, count discordant tasks, repeat k times
- The SQL grader **passes** when no SQL ran

> **Check the graders before the agents. A version that skips the tool gets a perfect SQL score.**

Source: notes/09-reading-results.md

Slice before you conclude

A flat headline can hide a loss

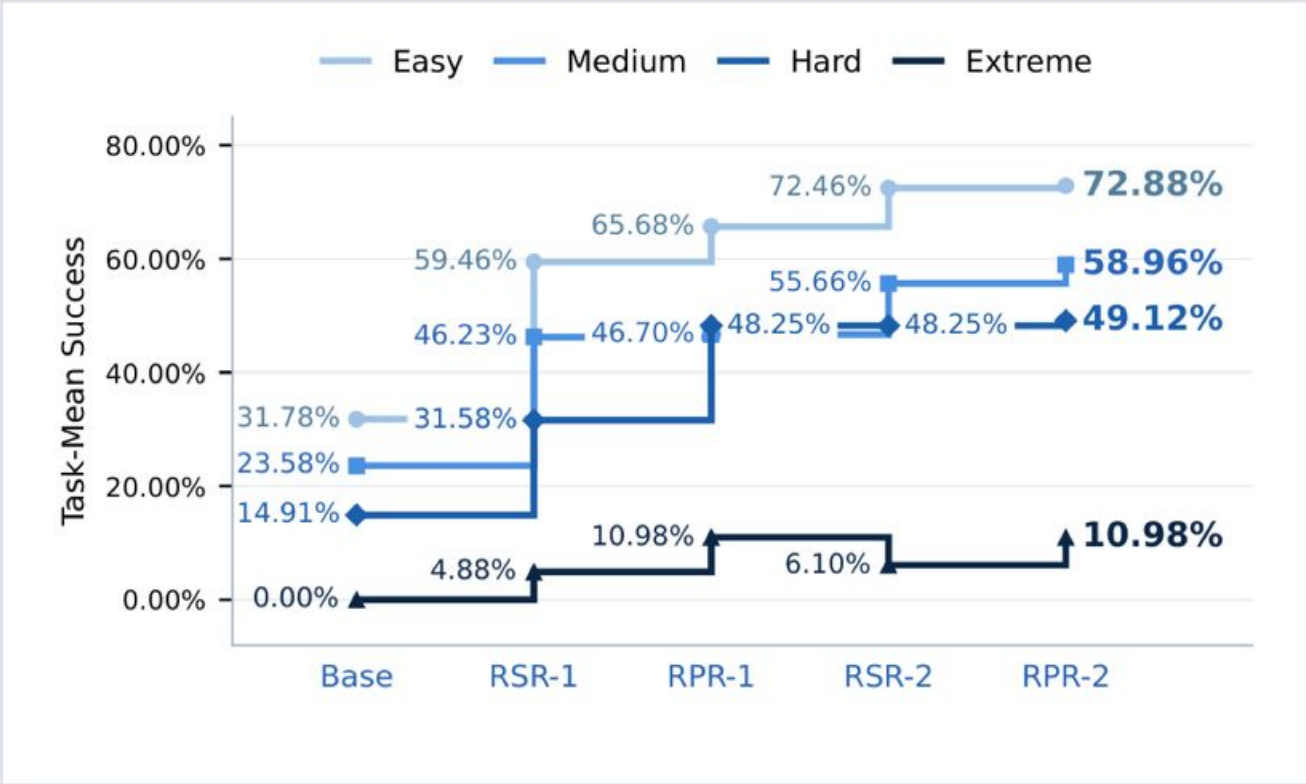
Slice	n	A	B	Δ
Billing	120	82%	90%	+8
How-to	150	88%	89%	+1
Non-English	40	75%	58%	**−17**
Overall	310	84%	85%	+1

> **"+1 overall" ships a change that breaks non-English users. Show n per slice.**

Source: notes/09-reading-results.md

Slice by difficulty, not just by topic

Yang et al. 2026, Terminal-Bench RSI



Success by difficulty stratum across self-improvement rounds (Yang et al. 2026, Fig. 10)

- **Easy** tasks: 32% → 73%. The headline gain lives here
- **Extreme** tasks: 0 → 4.9 → 11.0 → 6.1 → 11.0%
- One pooled rate hides where progress stalls

> **Report the strata. A rising average can sit on a flat hard tail.**

Source: notes/09-reading-results.md

A report that supports a decision

Report template

TEXT

```
Suite: support-golden v4 (n=310, 3 trials/task)   Judge: grounding-v2 (TNR 0.88)
                    A (prod)      B (candidate)   Δ (paired)
Pass rate            84.2% ±3.9    85.1% ±3.8      +0.9  [-1.6, +3.4]
 Non-English (40)  75.0% ±13     57.5% ±15       -17.5 [-29, -6]    ⚠
Errors / refusals    1.3%          4.2%                             ⚠
Cost per success     $0.014        $0.016          +14%
Flipped tasks read: 18 / 18. 9 of 11 regressions are refusal on non-English
Decision: do not ship. Investigate refusal behavior for non-English.
```

> **Count errors and refusals as failures, or as their own rate. Never let them leave the denominator.**

Source: notes/09-reading-results.md

Cheap suites on every change, expensive ones on a schedule

Trigger	Suite	Graders	Blocks?
Every commit / PR	Unit tests + smoke eval (10–30 tasks)	Code only	Yes
PR touching prompts, tools, retrieval	Regression suite	Code + calibrated judges on a sample	Yes
Nightly	Full suite, k trials	All, experimental tracked	Alerts
Pre-release	Full + held-out + adversarial	All calibrated	**Release gate**
Model change	Everything, paired vs. prod	All	Yes

> **Use path filters, so a README change doesn't trigger a $40 judge run.**

Source: notes/10-evals-in-ci.md

Run, then assert

Two phases with an artifact between

Phase 1: Run

- For each task × trial: run the SUT, capture the trace, grade
- Write a row to `results.jsonl`

Slow, costly, non-deterministic

Phase 2: Assert

- Load results, compute metrics and CIs per slice
- Compare to baseline and thresholds
- Emit pass/fail per gate + a report

Fast, free, deterministic

Change a threshold or add a slice and re-assert in seconds, without re-running a $30 suite.

Source: notes/10-evals-in-ci.md

Gate against a baseline, on the interval

Rule	Example	Catches
Absolute floor	Schema ≥ 99%. Citation hallucination = 0	Never-events
Relative to baseline	Grounding ≤ 3 points below prod (paired CI)	Regressions
Per-slice floor	No slice drops > 10 points	Hidden slice losses
Efficiency budget	Cost/success ≤ 1.2× baseline. p95 ≤ 8 s	Silent cost blow-ups
Regression set	Fixed failures ≥ 95% pass	Re-broken bugs

> **"Block if Δ < 0" blocks on noise half the time. Block when the CI upper bound is below –3.**

Source: notes/10-evals-in-ci.md

Gates ordered by pipeline stage



A later gate runs only on rows an earlier gate says are fair. The maturity ladder (next) orders building the practice; this orders checks within a run.

Source: notes/10-evals-in-ci.md

The maturity ladder

Where most systems sit

more signal

0 **0 · Vibes** — Manual review. Catches what you happen to look at.

1 **1 · Deterministic gates** — Tests, schemas, hooks. Highest return per effort.

2 **2 · Separated evaluator** — Fresh context, runs the output. Behavioral failures.

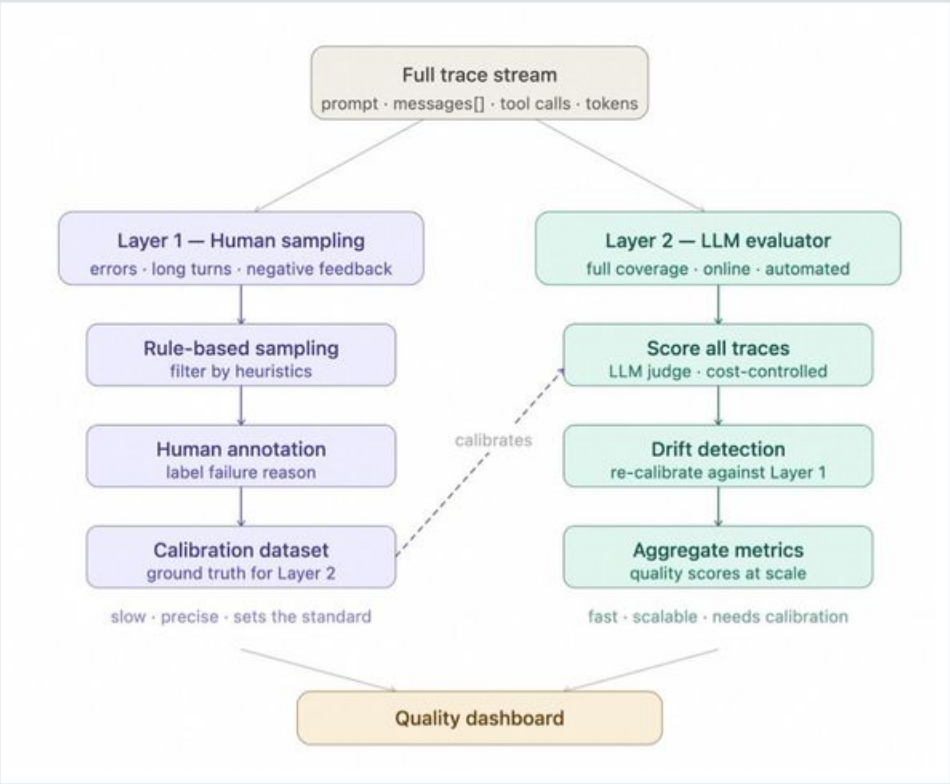
3 **3 · Failure evals + tracing** — Regressions, and where a run broke. **Production needs this.**

4 **4 · Continuous sampling** — Drift alerts against a launch baseline.

> **"We'll add evals later" means "we'll have no signal when this breaks".**

Source: notes/11-production-and-the-maturity-ladder.md

Humans calibrate, judges cover



Two-layer review of the production trace stream

- Grade **10–20%** of traces, chosen by rule
- **100%** of 👎, plus escalations, high cost, long chats
- A separate random slice for unbiased rates
- Full review for 48 h after any model or prompt change
- Track human–judge agreement as its own metric

> **Judges alone drift unnoticed. Humans alone don't cover real traffic.**

Source: notes/11-production-and-the-maturity-ladder.md

When a score drops, suspect the eval first

The debug order

1. **Infrastructure** — Errors, timeouts, 429s, OOM. Are error counts up?
2. **Versions** — Dataset, grader, judge model or config changed between runs?
3. **Traces + grades** — Read 10–20 failed transcripts **with** their grades. Is the grade right?
4. **Slices** — Is the drop concentrated in one category?
5. **Only then the agent** — Model, prompt, tools, logic.

> **Skipping to step 5 "fixes" an agent that wasn't broken, and sometimes breaks it.**

Source: notes/12-failure-modes.md

Optimize hard and you'll find the eval's gaps

Goodhart

Gap exploited	Example
Grader leniency	Answers padded until the verbosity-biased judge passes them
Narrow checks	Agent edits the tests, not the code. Valid schema, empty content
Proxy metric	👍 optimized → flattery. Escalations cut → won't escalate when it should
Visible tests	Overfits to the tests it can see

> **Defenses: several dimensions, held-out tests, read the biggest gains, refresh from production.**

Source: notes/12-failure-modes.md

Rules of thumb

The pillar on one slide

Wrap-up

Before any grader

Write success down. Two experts must agree. Read 50–100 traces first.

The dataset

Start with 20–50 real failures. Negatives and boundaries. Eligible rows only.

Graders

Code first. One judge per criterion. Uncalibrated judges never gate.

Agents and RAG

Outcome and trajectory. k trials, pass^k. Grounding claim by claim.

Reading and gating

Error bars, paired comparisons, slices. Run, then assert, against a baseline.

Production

Sample by rule. Humans calibrate the judges. Debug infra before the agent.

Source: README.md#rules-of-thumb

Worked example: which judges earned a gate

50 tasks, 25 liked and 25 disliked by users. Δ is liked minus disliked mean score.

Grader	Δ	Cohen's d	Verdict
Custom answer_relevancy v3	**+0.214**	0.75	Strong: ship
DeepEval answer_relevancy	+0.086	0.25	Weak
Custom completeness	+0.030	0.07	Marginal
DeepEval completeness	–0.042	—	**Inverted**
Custom escalation	0.000	0.00	No signal: don't gate
DeepEval escalation	–0.080	—	**Inverted**

> **Validate judges against users, not against other judges. Calibration doesn't transfer across domains.**

Source: notes/13-case-study-support-agent.md

Where to go next

Measures every layer below

- **Elo vs. Bradley–Terry:** worked example in 05, full lab upstream (ai-agent-book ch. 7)
- **Copy-ready snippets:** judge prompt (05), failure taxonomy (02), row schema (03), gate contract and CI workflow (10)
- **Worked example:** chapter 13, a real multi-agent support system's eval design, mistakes included

- **Course:** DeepLearning.AI, Evaluating AI Agents (Arize Phoenix), folded into 05, 06 and 08
- **Loops:** 04-loop-engineering, where an eval becomes the optimizer's target

The one idea to keep

An eval is a measurement. Define correct before grading, trust no grader you haven't checked against humans, and put an error bar on every number.

Source: README.md